

Chapter 4

C Program Control

An introduction to the control structures in C

Chapter Targets

- The essentials of counter-controlled iteration
- To use the **for** and **do...while** iteration statements to execute statements repeatedly
- To understand multiple selection using **switch** selection statements
- To use the **break** and **continue** statements to alter the flow of control
- To use the logical operators to form complex conditional expressions in control statements
- To avoid the consequences of confusing the equality and assignment operator

Introduction

- In this chapter
 - Iteration is considered in greater detail
 - Additional iteration control statements, namely the **for** and the **do...while**
 - **Switch** multiple-selection statement
 - **Break** statement for exiting immediately from certain control statements
 - **Continue** statement for skipping the remainder of the body of an iteration statement and proceeding with the next iteration of the loop.
 - Logical operators used for combining conditions
 - Summary of the principles of structured programming as presented in chapter 3 and 4.

Iteration Essentials

- A loop is a group of instructions the computer executes repeatedly while some **loop-continuation condition** remains true.
- We've discussed two means of iteration:
 - Counter-controlled iteration
 - Sentinel-controlled iteration
- **Counter-controlled iteration is sometimes called definite iteration** because we know in advance exactly how many times the loop will be executed.
- **Sentinel-controlled iteration is sometimes called indefinite iteration** because it's not known in advance how many times the loop will be executed.

Iteration Essentials (Cont'd)

- In counter-controlled iteration, a **control variable is used to count** the number of iterations.
- The control variable is incremented (usually by 1) each time the group of instructions is performed.
- When the value of the control variable indicates that the correct number of iterations has been performed, the loop terminates and execution continues with the statement after the iteration statement.

Iteration Essentials (Cont'd)

- Sentinel values are used to control iteration when:
 - The precise number of iterations isn't known in advance, and
 - The loop includes statements that obtain data each time the loop is performed.
- The **sentinel** value indicates “end of data.”
- The sentinel is entered after all regular data items have been supplied to the program.
- Sentinels **must be distinct from regular data** items.

Counter-Controlled Iteration

- Counter-controlled iteration requires:
 - The **name** of a control variable (or loop counter).
 - The **initial value** of the control variable.
 - The **increment** (or **decrement**) by which the control variable is modified each time through the loop.
 - The condition that tests for the **final value** of the control variable (i.e., whether looping should continue).

Counter-Controlled Iteration (Cont'd)

```
1 // Counter-controlled iteration
2 #include <stdio.h>
3
4 int main(void)
5 {
6     unsigned int counter = 1; // initialization
7
8     while (counter <= 10) { // iteration condition
9         printf ("%u\n", counter);
10        ++counter; // increment
11    }
12    return 0;
13 }
```

1
2
3
4
5
6
7
8
9
10

10
9
8

Counter-Controlled Iteration (Cont'd)

- The definition

```
unsigned int counter = 1; // initialization
```

names the control variable (counter), defines it to be an integer, reserves memory space for it, and sets it to an initial value of 1.

- This definition is not an executable statement.

Counter-Controlled Iteration (Cont'd)

- The definition and initialization of counter could also have been written as

```
unsigned int counter;  
counter = 1;
```

The definition is not executable, but the assignment is.

- We use both methods of setting the values of variables.
- The statement

```
++counter; // increment
```

increments the loop counter by 1 each time the loop is performed.

Counter-Controlled Iteration (Cont'd)

- The loop-continuation condition in the while statement tests whether the value of the control variable is less than or equal to 10 (the last value for which the condition is true).
- The body of this while is performed even when the control variable is 10.
- The loop terminates when the control variable exceeds 10 (i.e., counter becomes 11).

Counter-Controlled Iteration (Cont'd)

- You could make the program more concise by **initializing counter to 0** and by replacing the while statement with

```
while (++counter <= 10)
    printf("%u\n", counter);
```

This code saves a statement because the incrementing is done directly in the while condition before the condition is tested.

- Also, this code eliminates the need for the braces around the body of the while because the while now contains only one statement.
- Some programmers feel that this makes the code **too cryptic and error prone**.

Counter-Controlled Iteration (Cont'd)



Common Programming Error 4.1

Floating-point values may be approximate, so controlling counting loops with floating-point variables may result in imprecise counter values and inaccurate termination tests.



Error-Prevention Tip 4.1

Control counting loops with integer values.



Good Programming Practice 4.1

Too many levels of nesting can make a program difficult to understand. As a rule, try to avoid using more than three levels of nesting.



Good Programming Practice 4.2

The combination of vertical spacing before and after control statements and indentation of the bodies of control statements within the control-statement headers gives programs a two-dimensional appearance that greatly improves program readability.

for Iteration Statement

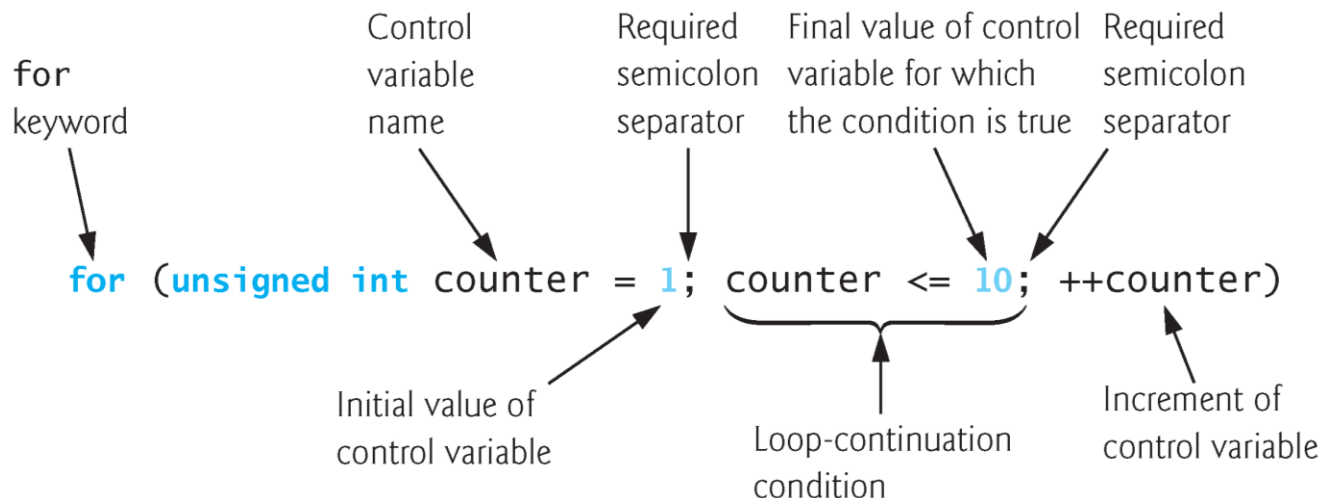
- The `for` iteration statement handles all the details of counter-controlled iteration.
- Let's rewrite the previous program.

```
1 // Counter-controlled iteration with the for statement
2 #include <stdio.h>
3
4 int main(void)
5 {
6     // initialization, iteration condition, and increment
7     // are all included in the for statement header.
8     for (unsigned int counter = 1; counter <= 10; ++counter) {
9         printf("%u\n", counter);
10    }
11    return 0;
12 }
```

for Iteration Statement (Cont'd)

- for Statement Header Components

- Notice that the for statement “does it all”—it specifies each of the items needed for counter-controlled iteration with a control variable.
- If there’s more than one statement in the body of the for, braces are required to define the body of the loop.



for Iteration Statement (Cont'd)

- When the for statement begins executing, the control variable counter is initialized to 1.
- Then, the loop-continuation condition `counter <= 10` is checked.
- Because the initial value of counter is 1, the condition is satisfied, so the `printf` statement (line 9) prints the value of counter, namely 1.
- The control variable counter is then incremented by the expression `++counter`, and the loop begins again with the loop-continuation test.

for Iteration Statement (Cont'd)

- Because the control variable is now equal to 2, the final value is not exceeded, so the program performs the `printf` statement again.
- This process continues until the control variable counter is incremented to its final value of 11—this causes the loop-continuation test to fail, and iteration terminates.
- The program continues by performing the first statement after the for statement (in this case, the end of the program).



Common Programming Error 4.2

For a control variable defined in a for statement's header, attempting to access the control variable after the for statement's closing right brace (}) is a compilation error.

for Iteration Statement (Cont'd)

- Off-By-One Errors (OBOE)

- Notice that the loop-continuation condition

`counter <= 10`

- If you incorrectly wrote `counter < 10`, then the loop would be executed only 9 times.
- This is a common logic error called an off-by-one error.



Error-Prevention Tip 4.2

Using the final value in the condition of a while or for statement and using the `<=` relational operator can help avoid off-by-one errors. For a loop used to print the values 1 to 10, for example, the loop-continuation condition should be `counter <= 10` rather than `counter < 11` or `counter < 10`.

for Iteration Statement (Cont'd)

- General Format of a for Statement

- The general format of the for statement is

```
for (initialization; condition; increment) {  
    statement  
}
```

where the initialization expression initializes the loop-control variable (and might define it), the condition expression is the loop-continuation condition and the increment expression increments the control variable.

for Iteration Statement (Cont'd)

- Comma-Separated Lists of Expressions
- Often, the initialization and increment expressions are comma-separated lists of expressions.
- The commas as used here are actually **comma operators** that guarantee that lists of expressions **evaluate from left to right**.
- The value and type of a comma-separated list of expressions are the value and type of the **rightmost expression** in the list.

for Iteration Statement (Cont'd)

- The comma operator is most often used in the for statement.
- Its primary use is to enable you to use multiple initialization and/or multiple increment expressions.
- For example, there may be two control variables in a single for statement that must be initialized and incremented.



Software Engineering Observation 4.1

Place only expressions involving the control variables in the initialization and increment sections of a for statement. Manipulations of other variables should appear either before the loop (if they execute only once, like initialization statements) or in the loop body (if they execute once per iteration, like incrementing or decrementing statements).

for Iteration Statement (Cont'd)

- Expressions in the for Statement's Header Are Optional
 - The three expressions in the for statement are optional.
 - If the condition expression is omitted, C assumes that the condition is true, thus creating an infinite loop.
 - You may omit the initialization expression if the control variable is initialized elsewhere in the program.
 - The increment may be omitted if it's calculated by statements in the body of the for statement or if no increment is needed.

for Iteration Statement (Cont'd)

- Increment Expression Acts Like a Standalone Statement

- The increment expression in the for statement acts like a stand-alone C statement at the end of the body of the for.

- Therefore, the expressions

```
counter = counter + 1  
counter += 1  
++counter  
counter++
```

are all equivalent in the increment part of the for statement.

- Because the variable being **preincremented** or **postincremented** here does not appear in a larger expression, both forms of incrementing **have the same effect**.
- The two semicolons in the for statement are required.

for Iteration Statement (Cont'd)



Common Programming Error 4.3

Using commas instead of semicolons in a for header is a syntax error.



Error-Prevention Tip 4.3

Infinite loops are caused when the loop-continuation condition in an iteration statement never becomes false. To prevent infinite loops, ensure that you do not place a semicolon immediately after a while statement's header. In a counter-controlled loop, make sure the control variable is incremented (or decremented) in the loop. In a sentinel-controlled loop, make sure the sentinel value is eventually input.

for Statement: Notes and Observations

- The initialization, loop-continuation condition and increment can contain arithmetic expressions. For example, if $x = 2$ and $y = 10$, the statement

`for (j = x; j <= 4 * x * y; j += y / x)`

is equivalent to the statement

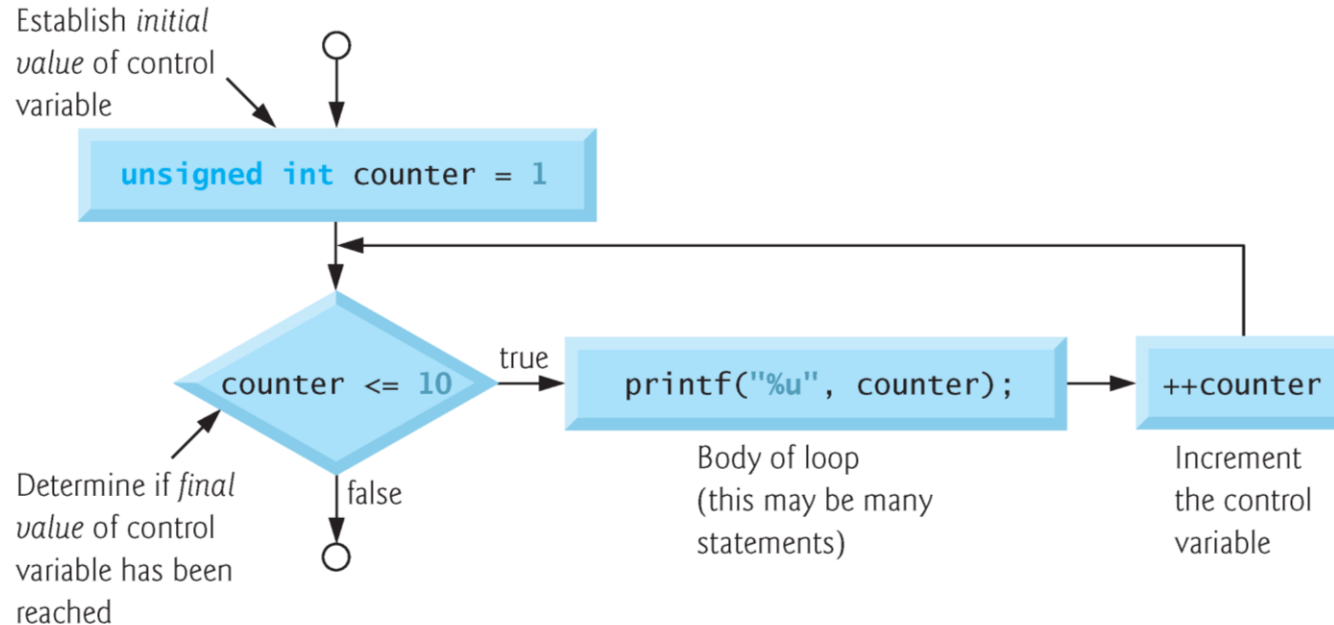
`for (j = 2; j <= 80; j += 5)`

- The “increment” may be negative (in which case it’s really a decrement and the loop actually counts downward).
- If the **loop-continuation** condition is initially **false**, the loop body does not execute. Instead, **execution proceeds with the statement following the for** statement.

for Statement: Notes and Observations (Cont'd)

- The for statement is flowcharted much like the while statement.

```
for (counter = 1; counter <= 10; ++counter)  
    printf("%u", counter);
```



for Statement: Notes and Observations (Cont'd)

- The control variable is frequently printed or used in calculations in the body of a loop, but it need not be. It's common to use the control variable for controlling iteration while never mentioning it in the body of the loop.



Error-Prevention Tip 4.4

Although the value of the control variable can be changed in the body of a for loop, this can lead to subtle errors. It's best not to change it.

Examples Using the for Statement

- The following examples show methods of varying the control variable in a for statement.
 - Vary the control variable from 1 to 100 in increments of 1.
`for (i = 1; i <= 100; ++i)`
 - Vary the control variable from 100 to 1 in increments of -1
`for (i = 100; i >= 1; --i)`
 - Vary the control variable from 7 to 77 in steps of 7.
`for (i = 7; i <= 77; i += 7)`
 - Vary the control variable from 20 to 2 in steps of -2.
`for (i = 20; i >= 2; i -= 2)`
 - Vary the control variable over the following sequence of values: 2, 5, 8, 11, 14, 17.
`for (j = 2; j <= 17; j += 3)`
 - Vary the control variable over the following sequence of values: 44, 33, 22, 11, 0.
`for (j = 44; j >= 0; j -= 11)`



Good Programming Practice 4.3

Limit the size of control-statement headers to a single line if possible.

Examples Using the for Statement (Cont'd)

- Application: Summing the Even Integers from 2 to 100

```
1 // Summation with for
2 #include <stdio.h>
3
4 int main(void)
5 {
6     unsigned int sum = 0; // initialize sum
7
8     for (unsigned int number = 2; number <= 100; number += 2) {
9         sum += number; // add number to sum
10    }
11
12    printf("Sum is %u\n", sum);
13    return 0;
14 }
```

Sum is 2550

Examples Using the for Statement (Cont'd)

- The body of the for statement could actually be merged into the rightmost portion of the for header by using the comma operator as follows:

```
for (number = 2; number <= 100; sum += number, number += 2)  
    ; // empty statement
```

- The initialization `sum = 0` could also be merged into the initialization section of the for.

Examples Using the for Statement (Cont'd)

- Application: Compound-Interest Calculations

- Consider the following problem statement:

- ◆ A person invests \$1000.00 in a savings account yielding 5% interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal)

r is the annual interest rate

n is the number of years

a is the amount on deposit at the end of the nth year.

- This problem involves a loop that performs the indicated calculation for each of the 10 years the money remains on deposit.

Examples Using the for Statement (Cont'd)

```
1 // Calculating compound interest
2 #include <stdio.h>
3 #include <math.h>
4
5 int main(void)
6 {
7     double principal = 1000.0; // start
8     double rate = .05; // annual interest
9
10    // output table column heads
11    printf("%4s%21s\n", "Year", "Amount on deposit");
12
13    // calculate amount on deposit for each of ten years
14    for (unsigned int year = 1; year <= 10; ++year) {
15
16        // calculate new amount for specified year
17        double amount = principal * pow(1.0 + rate, year);
18
19        // output one table row
20        printf("%4u%21.2f\n", year, amount);
21    }
22    return 0;
23 }
```

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Examples Using the for Statement (Cont'd)

- The for statement executes the body of the loop 10 times, varying a control variable from 1 to 10 in increments of 1.
- Although C does not include an exponentiation operator, we can use the [Standard Library function](#) `pow` for this purpose.
- The function `pow(x, y)` calculates the value of x raised to the y^{th} power.
- It takes two arguments of type `double` and returns a double value.
- Type [double is a floating-point type](#) like `float`, but typically a variable of type `double` can store a value of much [greater magnitude](#) with [greater precision than float](#).

Examples Using the for Statement (Cont'd)

- The header `<math.h>` (line 3) should be included whenever a **math function** such as `pow` is used.
- Actually, this program would malfunction without the inclusion of `math.h`, as the **linker would be unable to find the `pow` function**.
- Function `pow` requires two double arguments, but variable `year` is an integer.
- The `math.h` file includes information that tells the compiler to **convert the value of `year` to a temporary double** representation before calling the function.
- This information is contained in something called **`pow`'s function prototype**. (described in CH5)

Examples Using the for Statement (Cont'd)

- A Caution about Using Type `float` or `double` for Monetary Amounts
 - Notice that we defined the variables `amount`, `principal` and `rate` to be of type `double`.



Software Engineering Observation 4.2

Type `double` is a floating-point type like `float`, but typically a variable of type `double` can store a value of much greater magnitude with greater precision than `float`. Variables of type `double` occupy more memory than those of type `float`. For all but the most memory-intensive applications, professional programmers generally prefer `double` to `float`.



Error-Prevention Tip 4.5

Do not use variables of type `float` or `double` to perform monetary calculations. The impreciseness of floating-point numbers can cause errors that will result in incorrect monetary values. [In Exercise 4.23, we explore the use of integer numbers of pennies to perform precise monetary calculations.]

Examples Using the for Statement (Cont'd)

- Here is a simple explanation of what can go wrong when using float or double to represent dollar amounts.
 - Two float dollar amounts stored in the machine could be 14.234 (which with `%.2f` prints as 14.23) and 18.673 (which with `%.2f` prints as 18.67).
 - When these amounts are added, they produce the sum 32.907, which with `%.2f` prints as 32.91.
 - Thus your printout could appear as
$$\begin{array}{r} 14.23 \\ + 18.67 \\ \hline 32.91 \end{array}$$
 - Clearly the sum of the individual numbers as printed should be 32.90! You've been warned!

Examples Using the for Statement (Cont'd)

- Formatting Numeric Output

- The conversion specifier `%21.2f` is used to print the value of the variable amount in the program.
- The `21` in the conversion specifier denotes the `field width` in which the value will be printed.
- A field width of 21 specifies that the value printed will `appear in 21 print positions`.
- The `2` specifies the `precision` (i.e., the `number of decimal positions`).

Examples Using the for Statement.

- If the number of characters displayed is less than the field width, then the value will automatically be right justified in the field.
- This is particularly useful for aligning floating-point values with the same precision (so that their decimal points align vertically).
- To left justify a value in a field, place a - (minus sign) between the % and the field width.
- The minus sign may also be used to left justify integers (such as in %-6d) and character strings (such as in %-8s).

switch Multiple-Selection Statement

- Occasionally, an algorithm will **contain a series of decisions** in which a variable or expression is tested separately for each of the constant integral values it may assume, and different actions are taken.
- This is called **multiple selection**.
- C provides the **switch** multiple-selection statement to handle such decision making.
- The switch statement consists of a series of **case labels**, an **optional default case** and **statements to execute** for each case.

switch Multiple-Selection Statement (Cont'd)

```
1 // Counting letter grades with switch
2 #include <stdio.h>
3
4 int main(void)
5 {
6     unsigned int aCount = 0;
7     unsigned int bCount = 0;
8     unsigned int cCount = 0;
9     unsigned int dCount = 0;
10    unsigned int fCount = 0;
11
12    puts("Enter the letter grades." );
13    puts("Enter the EOF character to end input." );
14    int grade; // one grade
15
```

switch (Cont

ement

```
16 // loop until user types end-of-file key sequence
17 while ((grade = getchar()) != EOF) {
18
19     // determine which grade was input
20     switch (grade) { // switch nested in while
21
22         case 'A': // grade was uppercase A
23         case 'a': // or lowercase a
24             ++aCount;
25             break; // necessary to exit switch
26
27         case 'B': // grade was uppercase B
28         case 'b': // or lowercase b
29             ++bCount;
30             break;
31
32         case 'C': // grade was uppercase C
33         case 'c': // or lowercase c
34             ++cCount;
35             break;
36
37         case 'D': // grade was uppercase D
38         case 'd': // or lowercase d
39             ++dCount;
40             break;
41
42         case 'F': // grade was uppercase F
43         case 'f': // or lowercase f
44             ++fCount;
45             break;
46
47         case '\n': // ignore newlines,
48         case '\t': // tabs,
49         case ' ': // and spaces in input
50             break;
51
52         default: // catch all other characters
53             printf("%s", "Incorrect letter grade entered.");
54             puts(" Enter a new grade.");
55             break; // optional; will exit switch anyway
56     }
57 } // end while
58
```

switch Multiple-Selection Statement (Cont'd)

```
59 // output summary of results
60 puts("\nTotals for each letter grade are:");
61 printf("A: %u\n", aCount);
62 printf("B: %u\n", bCount);
63 printf("C: %u\n", cCount);
64 printf("D: %u\n", dCount);
65 printf("F: %u\n", fCount);
66 return 0;
67 }
```

```
Enter the letter grades.
Enter the EOF character to end input.
a
b
c
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
b
^Z ——— Not all systems display a representation of the EOF character

Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1
```

switch Multiple-Selection Statement (Cont'd)

- Reading Character Input

- In the program, the user enters letter grades for a class.
- In the while header (line 17),
`while ((grade = getchar()) != EOF)`
the parenthesized assignment (`grade = getchar()`) executes first.
- The `getchar` function (from `<stdio.h>`) reads one character from the keyboard and returns as an `int` the character that the user entered.
- Characters are normally stored in variables of type `char`.
- However, an important feature of C is that characters can be stored in any integer data type because they're usually represented as one-byte integers in the computer.

switch Multiple-Selection Statement (Cont'd)

- Thus, we can treat a character as either an integer or a character, depending on its use.
- For example, the statement

```
printf("The character (%c) has the value %d.\n", 'a', 'a');
```
- uses the conversion specifiers `%c` and `%d` to print the character `a` and its integer value, respectively.
- The result is
The character (a) has the value 97.
- The integer 97 is the character's numerical representation in the computer.

switch Multiple-Selection Statement (Cont'd)

- Many computers today use the **ASCII (American Standard Code for Information Interchange)** character set in which 97 represents the lowercase letter 'a'.
- A list of the ASCII characters and their decimal values is presented in **Appendix B**.
- Characters can be read with `scanf` by using the conversion specifier `%c`.
- **Assignments as a whole actually have a value.**
- This value is assigned to the variable on the left side of `=`.
- The value of the assignment expression `grade = getchar()` is the character that's returned by `getchar` and assigned to the variable `grade`.

switch Multiple-Selection Statement (Cont'd)

- The fact that assignments have values can be useful for setting several variables to the same value.
- For example, `a = b = c = 0;` first evaluates the assignment `c = 0` (because the `=` operator associates from right to left).
- The variable `b` is then assigned the value of the assignment `c = 0` (which is 0).
- Then, the variable `a` is assigned the value of the assignment `b = (c = 0)` (which is also 0).
- In the program, the value of the assignment `grade = getchar()` is compared with the value of `EOF` (a symbol whose acronym stands for “end of file”).



Portability Tip 4.1

The keystroke combinations for entering EOF (end of file) are system dependent.

switch Multiple-Selection Statement (Cont'd)

- We use `EOF` (which normally has the value `-1`) as the **sentinel value**.
- The user types a system-dependent keystroke combination to mean “end of file”—i.e., “I have no more data to enter.” `EOF` is **a symbolic integer constant** defined in the `<stdio.h>` header (CH6).
- If the value assigned to `grade` is equal to `EOF`, the program terminates.
- We’ve chosen to represent characters in this program as ints because `EOF` has an integer value (normally `-1`).



Portability Tip 4.2

Testing for the symbolic constant `EOF` (rather than `-1`) makes programs more portable. The C standard states that `EOF` is a negative integral value (but not necessarily `-1`). Thus, `EOF` could have different values on different systems.

switch Multiple-Selection Statement (Cont'd)

- Entering the EOF Indicator

- On Linux/UNIX/Mac OS X systems, the EOF indicator is entered by typing

`<Ctrl> d`

on a line by itself.

- This notation `<Ctrl> d` means to press the Enter key then simultaneously press both Ctrl and d.
- On other systems, such as Microsoft Windows, the EOF indicator can be entered by typing

`<Ctrl> z`

- You may also need to press Enter on Windows.

switch Multiple-Selection Statement (Cont'd)

- The user enters grades at the keyboard.
- When the Enter key is pressed, the characters are read by function `getchar` one character at a time.
- If the character entered is not equal to EOF, the switch statement (line 20) is entered.

switch Multiple-Selection Statement (Cont'd)

- `switch` Statement Details

- Keyword `switch` is followed by the variable name `grade` in parentheses.
- This is called the **controlling expression**.
- The value of this expression is **compared with each of the case labels**.
- Assume the user has entered the letter `C` as a grade.
- `C` is automatically compared to each case in the `switch`.
- **If a match occurs (case `'C':`), the statements for that case are executed.**

switch Multiple-Selection Statement (Cont'd)

- In the case of the letter C, `cCount` is incremented by 1 (line 34), and the switch statement is exited immediately with the `break` statement.
- Listing several case labels together (such as `case 'C' : case 'c' :`) simply means that the same set of actions is to occur for either of these cases.
- The **`break`** statement causes program control to `continue` with the first statement `after the switch` statement.
- The **`break`** statement is used because the cases in a switch statement would otherwise run together.

switch Multiple-Selection Statement (Cont'd)

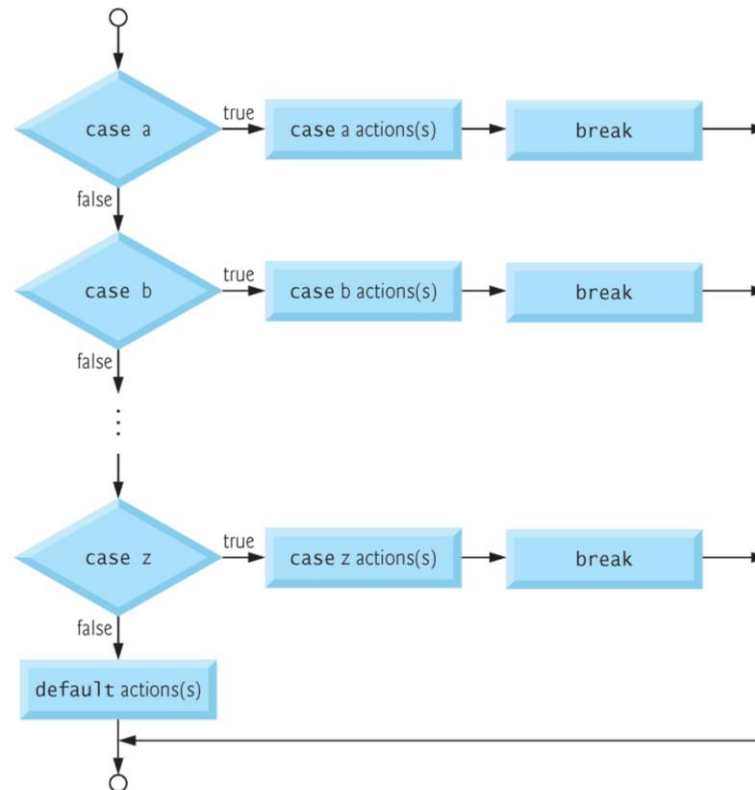
- If **break is not used** anywhere in a switch statement, then each time **a match occurs** in the statement, the statements for **all the remaining cases will be executed**—called **fall through**.
- If **no match** occurs, the **default case is executed**, and an **error message** is printed.

switch Multiple-Selection Statement (Cont'd)

- `switch` Statement Flowchart
 - Each case can have one or more actions.
 - The switch statement is different from all other control statements in that **braces are not required around multiple actions** in a case of a switch.

switch Multiple-Selection Statement (Cont'd)

- Each break statement at the end of a case causes control to immediately exit the switch statement.



switch Multiple-Selection Statement (Cont'd)



Common Programming Error 4.4

Forgetting a break statement when one is needed in a switch statement is a logic error.



Error-Prevention Tip 4.6

Provide a default case in switch statements. Values not explicitly tested in a switch would normally be ignored. The default case helps prevent this by focusing you on the need to process exceptional conditions. Sometimes no default processing is needed.



Good Programming Practice 4.4

Although the case clauses and the default case clause in a switch statement can occur in any order, it's common to place the default clause last.



Good Programming Practice 4.5

In a switch statement, when the default clause is last, the break statement isn't required. You may prefer to include this break for clarity and symmetry with other cases.

switch Multiple-Selection Statement (Cont'd)

- Ignoring Newline, Tab and Blank Characters in Input
 - In the switch statement, the lines

```
case '\n': // ignore newlines,  
case '\t': // tabs,  
case ' ': // and spaces in input  
break; // exit switch
```

cause the program to skip newline, tab and blank characters.
 - Reading characters one at a time can cause some problems.
 - To have the program read the characters, you must send to the computer by pressing the Enter key.
 - This causes the newline character to be placed in the input after the character we wish to process.

switch Multiple-Selection Statement (Cont'd)

- Often, this newline character must be specially processed to make the program work correctly.
- By including the preceding cases in our switch statement, we prevent the error message in the default case from being printed each time a newline, tab or space is encountered in the input.
- So each input causes **two iterations** of the loop—the first for the **letter grade** and the second for **'\n'**.



Error-Prevention Tip 4.7

Remember to provide processing capabilities for newline (and possibly other white-space) characters in the input when processing characters one at a time.

switch Multiple-Selection Statement (Cont'd)

- Constant Integral Expressions

- When using the switch statement, remember that each individual case can test only a constant integral expression—i.e., any combination of character constants and integer constants that evaluates to a constant integer value.
- A character constant can be represented as the specific character in single quotes, such as 'A'. Note that characters in double quotes are recognized as strings.
- Integer constants are simply integer values.
- Remember that characters are represented as small integer values.

switch Multiple-Selection Statement (Cont'd)

- Notes on Integral Types

- Portable languages like C must have flexible data type sizes.
- Different applications may need integers of different sizes.
- C provides several data types to represent integers.
- In addition to `int` and `char`, C provides types `short int` (which can be abbreviated as `short`) and `long int` (which can be abbreviated as `long`), as well as `unsigned` variations of all the integral types.
- The C standard specifies the minimum range of values for each integer type, but the actual range may be greater and depends on the implementation.
- For short ints the minimum range is -32767 to $+32767$.
- For most integer calculations, long ints are sufficient.

switch Multiple-Selection Statement (Cont'd)

- The minimum range of values for long ints is – 2147483647 to +2147483647.
- The range of values for an int greater than or equal to that of a short int and less than or equal to that of a long int.
- The data type signed char can be used to represent integers in the range –127 to +127 or any of the characters in the computer's character set.

do...while Iteration Statement

- The **do...while** iteration statement is similar to the while statement.
- In the while statement, the loop-continuation condition is tested at the beginning of the loop before the body of the loop is performed.
- The do...while statement tests the loop-continuation condition after the loop body is performed.
- Therefore, the loop body will be executed at least once.
- When a **do...while** terminates, execution continues with the statement after the while clause.

do...while Iteration Statement (Cont'd)

- It's not necessary to use braces in the do...while statement if there's only one statement in the body.
- However, the braces are usually included to avoid confusion between the while and do...while statements.
- For example,

```
while (condition)
```
- is normally regarded as the header to a while statement.

do...while Iteration Statement (Cont'd)

- A do...while with **no braces** around the single-statement body appears as

```
do
    statement
while (condition);
```

which **can be confusing**.

- The last line—while(condition);—may be misinterpreted as a while statement containing an empty statement.
- Thus, to avoid confusion, the do...while with one statement is often written as follows:

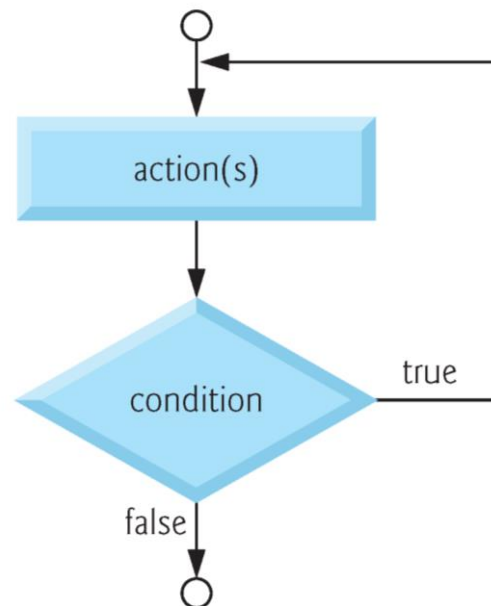
do...while Iteration Statement (Cont'd)

```
1 // Using the do...while iteration statement
2 #include <stdio.h>
3
4 int main(void)
5 {
6     unsigned int counter = 1;
7
8     do {
9         printf("%u  ", counter);
10    } while (++counter <= 10);
11    return 0;
12 }
```

1 2 3 4 5 6 7 8 9 10

do...while Iteration Statement.

- The **do...while** statement flowchart makes it clear that the loop-continuation condition does not execute until after the action is performed at least once.



break and continue Statements

- The **break** and **continue** statements are used to **alter the flow of control**.
- **break** Statement
 - The `break` statement, when executed in a while, for, do...while or switch statement, **causes an immediate exit from that statement**.
 - Program execution **continues with the next statement**.
 - Common uses of the break statement are **to escape early from a loop or to skip the remainder of a switch statement**.

break and continue Statements (Cont'd)

- When the if statement detects that x has become 5, break is executed.
- This terminates the for statement, and the program continues with the printf after the for.
- The loop fully executes only four times.

```
1 // Using the break statement in a for statement
2 #include <stdio.h>
3
4 int main(void)
5 {
6     unsigned int x; // declared here so it can be used
7
8     // loop 10 times
9     for (x = 1; x <= 10; ++x) {
10
11         // if x is 5, terminate loop
12         if (x == 5) {
13             break; // break loop only if x is 5
14         }
15
16         printf("%u ", x);
17     }
18
19     printf("\nBroke out of loop at x == %u\n", x);
20     return 0;
21 }
```

```
1 2 3 4
Broke out of loop at x == 5
```

break and continue Statements (Cont'd)

● **continue** Statement

- The **continue** statement, when executed in a while, for or do...while statement, **skips the remaining statements in the body** of that control statement **and performs the next iteration** of the loop.
- In while and do...while statements, **the loop-continuation test is evaluated immediately after the continue statement is executed.**
- In the for statement, **the increment expression is executed, then the loop-continuation test is evaluated.**

break and continue Statements (Cont'd)

```
1 // Using the continue statement in a for statement
2 #include <stdio.h>
3
4 int main(void)
5 {
6     // loop 10 times
7     for (unsigned int x = 1; x <= 10; ++x) {
8
9         // if x is 5, continue with next iteration of loop
10        if (x == 5) {
11            continue; // skip remaining code in loop body
12        }
13
14        printf("%u ", x); // display value of x
15    }
16
17    puts("\nUsed continue to skip printing the value 5");
18    return 0;
19 }
```

1 2 3 4 6 7 8 9 10
Used continue to skip printing the value 5

break and continue Statements (Cont'd)



Software Engineering Observation 4.3

Some programmers feel that break and continue violate the norms of structured programming. The effects of these statements can be achieved by structured programming techniques we'll soon discuss, so these programmers do not use break and continue.



Performance Tip 4.1

The break and continue statements, when used properly, perform faster than the corresponding structured techniques that we'll soon learn.



Software Engineering Observation 4.4

There's a tension between achieving quality software engineering and achieving the best-performing software. Often one of these goals is achieved at the expense of the other. For all but the most performance-intensive situations, apply the following guidelines: First, make your code simple and correct; then make it fast and small, but only if necessary.

Logical Operators

- C provides logical operators that may be used to form more complex conditions by combining simple conditions.
- The logical operators are `&&` (logical AND), `||` (logical OR) and `!` (logical NOT also called logical negation).

Logical Operators (Cont'd)

- Logical AND (& &) Operator

- Suppose we wish to ensure that two conditions are **both true** before we choose a certain path of execution.
- In this case, we can use the logical operator & & as follows:

```
if (gender == 1 && age >= 65)
    ++seniorFemales;
```

- This if statement contains two simple conditions.
 - ◆ The condition `gender == 1` might be evaluated, for example, to determine if a person is a female.
 - ◆ The condition `age >= 65` is evaluated to determine whether a person is a senior citizen.
- The two simple **conditions are evaluated first** because the precedence of `==` and `>=` are both higher than the precedence of `&&`.

Logical Operators (Cont'd)

- The if statement then considers the combined condition

```
gender == 1 && age >= 65
```

Which is true if and only if both of the simple conditions are true.

- Finally, if this combined condition is true, then the count of `seniorFemales` is incremented by 1.
- If either or both of the simple conditions are false, then the program skips the incrementing and proceeds to the statement following the if.

Logical Operators (Cont'd)

- Truth table of `&` & `&&`

expression1	expression2	expression1 && expression2
0	0	0
0	nonzero	0
nonzero	0	0
nonzero	nonzero	1

- C evaluates all expressions that include relational operators, equality operators, and/or logical operators to 0 or 1.
- Although C sets a true value to 1, it accepts any nonzero value as true.

Logical Operators (Cont'd)

- Logical OR (||) Operator

- Now let's consider the || (logical OR) operator.
- Suppose we wish to ensure at some point in a program that **either or both of two conditions are true** before we choose a certain path of execution.
- In this case, we use the || operator as in the following program segment

```
if (semesterAverage >= 90 || finalExam >= 90)
    printf("Student grade is A");
```

This statement also contains two simple conditions.

- The condition `semesterAverage >= 90` is evaluated to determine whether the student deserves an “A” in the course because of a solid performance throughout the semester.

Logical Operators (Cont'd)

- The condition `finalExam >= 90` is evaluated to determine whether the student deserves an “A” in the course because of an outstanding performance on the final exam.
- The `if` statement then considers the combined condition

`semesterAverage >= 90 || finalExam >= 90`

and awards the student an “A” if either or both of the

- The only (zero)

expression1	expression2	expression1 expression2
0	0	0
0	nonzero	1
nonzero	0	1
nonzero	nonzero	1

nted
false

Logical Operators (Cont'd)

- The `&&` operator has a higher precedence than `||`.
- Both operators **associate from left to right**.
- An expression containing `&&` or `||` operators is **evaluated only until truth or falsehood is known**.
- Thus, evaluation of the condition

```
gender == 1 && age >= 65
```

will stop if gender is not equal to 1, and continue if gender is equal to 1 (i.e., the entire expression could still be true if age `>= 65`).
- This performance feature for the evaluation of logical AND and logical OR expressions is called **short-circuit evaluation**.



Performance Tip 4.2

In expressions using operator `&&`, make the condition that's most likely to be false the leftmost condition. In expressions using operator `||`, make the condition that's most likely to be true the leftmost condition. This can reduce a program's execution time.

Logical Operators (Cont'd)

expression	! expression
0	1
nonzero	0

- Logical Negation (!) Operator

- C provides ! (logical negation) to enable you to “reverse” the meaning of a condition.
- The logical negation operator has only a single condition as an operand (and is therefore a unary operator).
- Placed before a condition when we’re interested in choosing a path of execution if the original condition (without the logical negation operator) is false, such as in the following program segment:

```
if (!(grade == sentinelValue))  
    printf("The next grade is %f\n", grade);
```

- The parentheses around the condition `grade == sentinelValue` are needed because the logical negation operator has a higher precedence than the equality operator.

Logical Operators (Cont'd)

- In most cases, you can avoid using logical negation by expressing the condition differently with an appropriate relational operator.
- For example, the preceding statement may also be written as follows:

```
if (grade != sentinelValue)
    printf("The next grade is %f\n", grade);
```

Logical Operators (Cont'd)

- Summary of Operator Precedence and Associativity

Operators	Associativity	Type
<code>++</code> (<i>postfix</i>) <code>--</code> (<i>postfix</i>)	right to left	postfix
<code>+</code> <code>-</code> <code>!</code> <code>++</code> (<i>prefix</i>) <code>--</code> (<i>prefix</i>) (<i>type</i>)	right to left	unary
<code>*</code> <code>/</code> <code>%</code>	left to right	multiplicative
<code>+</code> <code>-</code>	left to right	additive
<code><</code> <code><=</code> <code>></code> <code>>=</code>	left to right	relational
<code>==</code> <code>!=</code>	left to right	equality
<code>&&</code>	left to right	logical AND
<code> </code>	left to right	logical OR
<code>?:</code>	right to left	conditional
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	right to left	assignment
<code>,</code>	left to right	comma

Logical Operators.

- The `_Bool` Data Type

- The C standard includes a boolean type—represented by the keyword `_Bool`—which can hold only the values 0 or 1.
- Recall C's convention of using zero and nonzero values to represent false and true—the value 0 in a condition evaluates to false, while any nonzero value evaluates to true.
- Assigning any non-zero value to a `_Bool` sets it to 1.
- The standard also includes the `<stdbool.h>` header, which defines `bool` as a shorthand for the type `_Bool`, and `true` and `false` as named representations of 1 and 0, respectively.
- At preprocessor time, `bool`, `true` and `false` are replaced with `_Bool`, 1 and 0.

Confusing Equality (==) and Assignment (=) Operators

- There's one type of error that C programmers, no matter how experienced, tend to make so frequently that we felt it was worth a separate section.
- That error is **accidentally swapping** the operators **==** (equality) and **=** (assignment).
- What makes these swaps so damaging is the fact that they **do not** ordinarily **cause compilation errors**.
- Rather, statements with these errors ordinarily compile correctly, allowing programs to **run to completion** while likely generating incorrect results through **runtime logic errors**.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- Two aspects of C cause these problems.
 - One is that **any expression** in C that **produces a value can be used in the decision** portion of any control statement.
 - If the value is 0, it's treated as false, and if the value is nonzero, it's treated as true.
 - The second is that **assignments in C produce a value**, namely the value that's assigned to the variable on the left side of the assignment operator.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- For example, suppose we intend to write

```
if (payCode == 4)
    printf("%s", "You get a bonus!");
```

but we accidentally write

```
if (payCode = 4)
    printf("%s", "You get a bonus!");
```

- The first **if** statement properly awards a bonus to the person whose `payCode` is equal to 4.
- The second **if** statement—the one with the error—evaluates the assignment expression in the **if** condition.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- This expression is a simple assignment whose value is the constant 4.
- Because any nonzero value is interpreted as “true,” **the condition in this if statement is always true**, and not only is the value of `payCode` inadvertently set to 4, but the person always receives a bonus regardless of what the actual `payCode` is!



Common Programming Error 4.5

Using operator == for assignment or using operator = for equality is a logic error.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- lvalues and rvalues

- You'll probably be inclined to write conditions such as `x == 7` with the variable name on the left and the constant on the right.
- By reversing these terms so that the constant is on the left and the variable name is on the right, as in `7 == x`, then if you accidentally replace the `==` operator with `=`, you'll be protected by the compiler.
- The compiler will treat this as a **syntax error**, because only a variable name can be placed on the left-hand side of an assignment expression.
- This will **prevent** the potential devastation of a **runtime logic error**.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- Variable names are said to be **lvalues** (for “left values”) because they **can be used on the left side** of an assignment operator.
- Constants are said to be **rvalues** (for “right values”) because **they can be used on only the right side** of an assignment operator.
- **lvalues can also be used as rvalues, but not vice versa.**



Error-Prevention Tip 4.8

When an equality expression has a variable and a constant, as in `x == 1`, you may prefer to write it with the constant on the left and the variable name on the right (i.e., `1 == x`) as protection against the logic error that occurs when you accidentally replace operator `==` with `=`.

Confusing Equality (==) and Assignment (=) Operators (Cont'd)

- Confusing == and = in Standalone Statements
 - The other side of the coin can be equally unpleasant.
 - Suppose you want to assign a value to a variable with a simple statement such as
`x = 1;`
but instead write
`x == 1;`
 - Here, too, this is not a syntax error.
 - Rather the compiler simply evaluates the conditional expression.

Confusing Equality (==) and Assignment (=) Operators.

- If x is equal to 1, the condition is true and the expression returns the value 1.
- If x is not equal to 1, the condition is false and the expression returns the value 0.
- Regardless of what value is returned, **there's no assignment operator, so the value is simply lost**, and the value of x remains unaltered, probably causing an execution-time logic error.
- Unfortunately, we do not have a handy trick available to help you with this problem! Many compilers, however, will issue a **warning** on such a statement.



Error-Prevention Tip 4.9

After you write a program, text search it for every = and check that it's used properly. This can help you prevent subtle bugs.

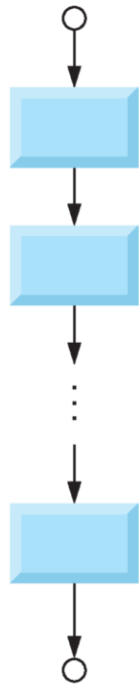
Structured Programming Summary

- Small circles are used in the figure to indicate the single entry point and the single exit point of each statement.
- Connecting individual flowchart symbols arbitrarily can lead to unstructured programs.
- Therefore, the programming profession has chosen to combine flowchart symbols to form a limited set of control statements, and to build only structured programs by properly combining control statements in two simple ways.

Structured Programming Summary (Cont'd)

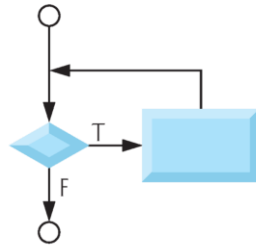
Sequence

Sequence

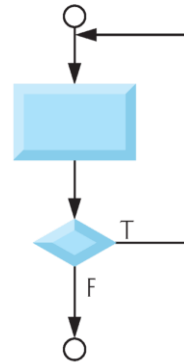


Repetition

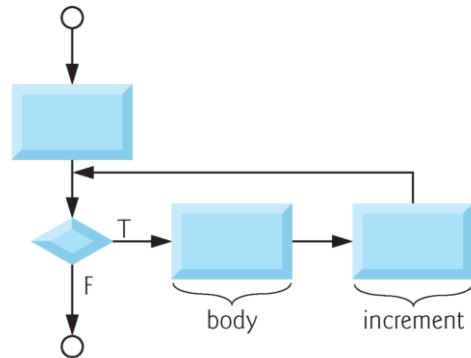
while statement



do...while statement

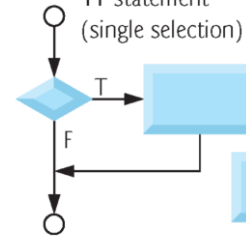


for statement

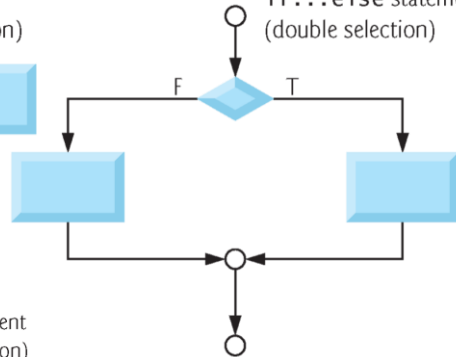


Selection

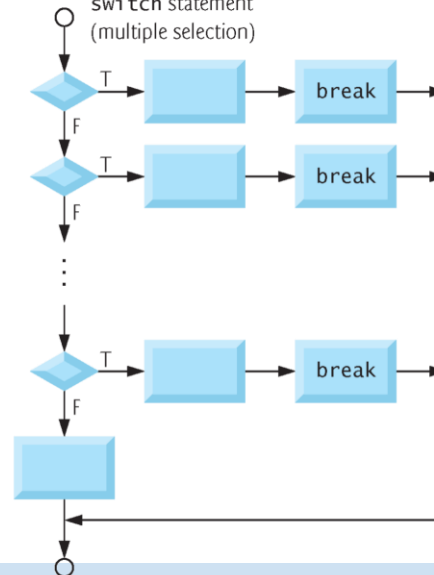
if statement
(single selection)



if...else statement
(double selection)



switch statement
(multiple selection)



Structured Programming Summary (Cont'd)

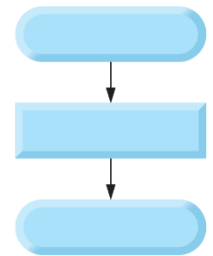
- For simplicity, **only single-entry/single-exit control statements are used**—there's only one way to enter and only one way to exit each control statement.
- Connecting control statements in sequence to form structured programs is simple—the **exit point of one control statement is connected directly to the entry point of the next**, i.e., the control statements are simply placed one after another in a program—we've called this "**control-statement stacking**."
- The rules for forming structured programs also allow for control statements to be nested.

Structured Programming Summary (Cont'd)

- Rules for Forming Structured Programs
 - The rectangle flowchart symbol may be used to indicate any action including input/output.
 - Applying the rules always results in a structured flowchart with a neat, building-block appearance.

Rules for forming structured programs

1. Begin with the “simplest flowchart”
2. (“Stacking” rule) Any rectangle (action) can be replaced by *two* rectangles (actions) in sequence.
3. (“Nesting” rule) Any rectangle (action) can be replaced by *any* control statement (sequence, if, if...else, switch, while, do...while or for).
4. Rules 2 and 3 may be applied as often as you like and in *any* order.

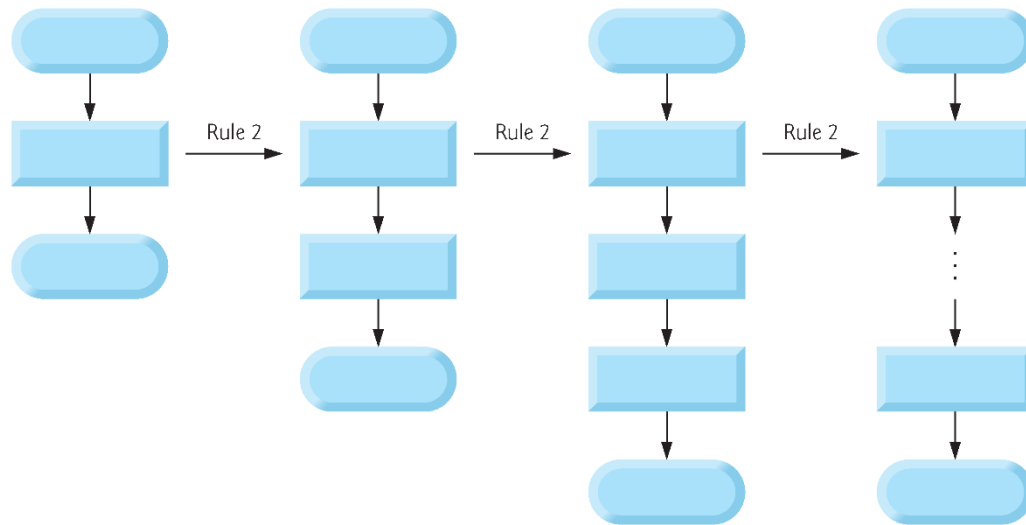


**Simplest
flowchart**

Structured Programming Summary (Cont'd)

■ Stacking rule

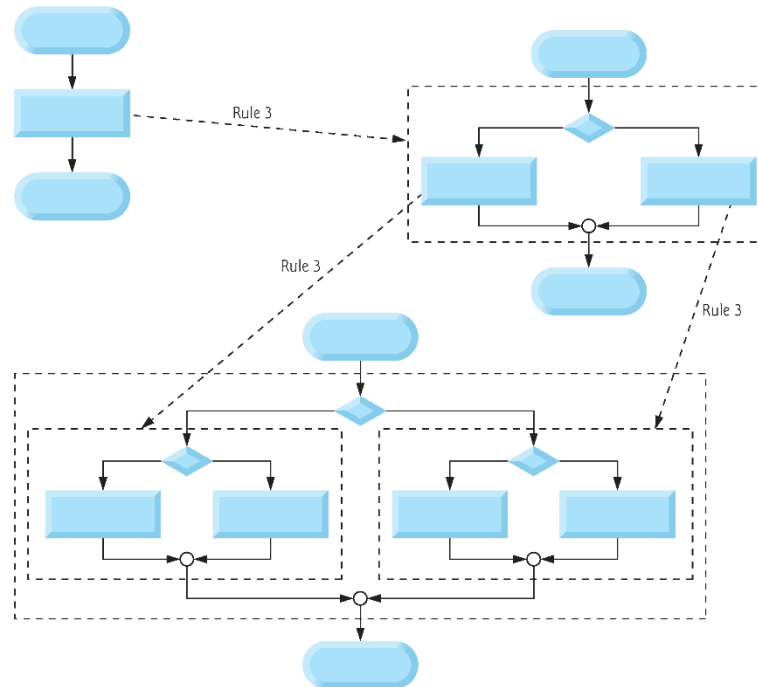
- ◆ Repeatedly applying Rule 2 to the simplest flowchart results in a structured flowchart containing many rectangles in sequence.
- ◆ Rule 2 generates a stack of control statements; so we call Rule 2 the stacking rule.



Structured Programming Summary (Cont'd)

■ Nesting rule

- ◆ Repeatedly applying Rule 3 to the simplest flowchart results in a flowchart with neatly nested control statements.

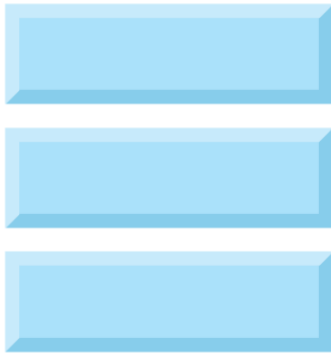


Structured Programming Summary (Cont'd)

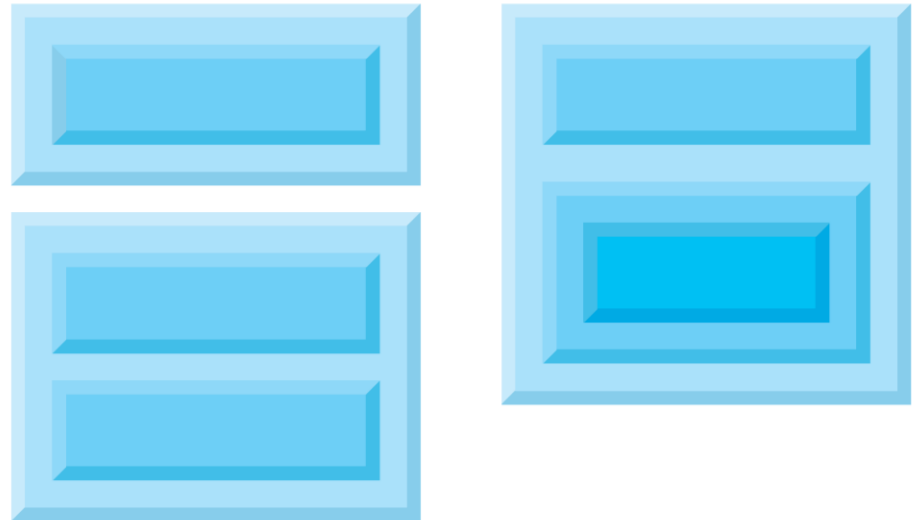
- Rule 4 generates larger, more involved, and more deeply nested structures.
- The flowcharts that emerge from applying the rules constitute the set of all possible structured flowcharts and hence the set of all possible structured programs.
- It's because of the **elimination of the goto** statement that these **building blocks never overlap** one another.
- The beauty of the structured approach is that we **use only a small number of simple single-entry/single-exit pieces**, and we **assemble them in only two simple ways**.

Structured Programming Summary (Cont'd)

Stacked building blocks



Nested building blocks



Overlapping building blocks
(Illegal in structured programs)



Structured Programming Summary (Cont'd)

- If the rules are followed, an unstructured flowchart cannot be created.
- If you're uncertain whether a particular flowchart is structured, apply the rules in reverse to try to reduce the flowchart to the simplest flowchart.
- If you succeed, the original flowchart is structured; otherwise, it's not.
- Structured programming promotes simplicity.
- Bohm and Jacopini showed that only three forms of control are needed:
 - Sequence
 - Selection
 - Iteration

Structured Programming Summary (Cont'd)

- Sequence is straightforward.
- Selection is implemented in one of three ways:
 - if statement (single selection)
 - if...else statement (double selection)
 - switch statement (multiple selection)
- In fact, it's straightforward to prove that the simple if statement is sufficient to provide any form of selection—everything that can be done with the if...else statement and the switch statement can be implemented with one or more if statements.

Structured Programming Summary (Cont'd)

- Iteration is implemented in one of three ways:
 - while statement
 - do...while statement
 - for statement
- It's straightforward to prove that the **while statement is sufficient to provide any form of iteration.**
- Everything that can be done with the do...while statement and the for statement can be done with the while statement.

Structured Programming Summary (Cont'd)

- Combining these results illustrates that any form of control ever needed in a C program can be expressed in terms of only three forms of control:
 - sequence
 - if statement (selection)
 - while statement (iteration)
- And these control statements can be combined in only two ways—**stacking and nesting**.
- Indeed, structured programming promotes simplicity.

Structured Programming Summary (Cont'd)

- In Chapters 3 and 4, we discussed how to compose programs from control statements containing actions and decisions.
- In Chapter 5, we introduce another program structuring unit called the **function**.
 - We'll learn to compose large programs by combining functions, which, in turn, are composed of control statements.
 - We'll also discuss how using functions promotes software **reusability**.

Secure C Programming

- Checking Function `scanf`'s Return Value
 - Many functions return values indicating whether they executed successfully.
 - For example, function `scanf` returns an int indicating whether the input operation was successful.
 - If an input **failure** occurs, `scanf` returns the value **EOF** (defined in `<stdio.h>`); otherwise, it returns **the number of items that were read**.
 - If this value does not match the number you intended to read, then `scanf` was unable to complete the input operation.

Secure C Programming (Cont'd)

- Consider the following statement

```
scanf("%d", &grade); // read grade from user
```

which expects to read one int value.

- If the user enters an integer, scanf returns 1 indicating that one value was indeed read.
- If the user enters a string, such as "hello", scanf returns 0 indicating that it was unable to read the input as an integer.
- In this case, the variable grade does not receive a value.

Secure C Programming (Cont'd)

- Function `scanf` can read multiple inputs, as in

```
scanf("%d%d", &number1, &number2); // read two integers
```
- If the input is successful, `scanf` will return 2 indicating that two values were read.
- If the user enters a string for the first value, `scanf` will return 0 and neither `number1` nor `number2` will receive values.
- If the user enters an integer followed by a string, `scanf` will return 1 and only `number1` will receive a value.



Error-Prevention Tip 4.10

To make your input processing more robust, check `scanf`'s return value to ensure that the number of inputs read matches the number of inputs expected. Otherwise, your program will use the values of the variables as if `scanf` completed successfully. This could lead to logic errors, program crashes or even attacks.

Secure C Programming.

- Range Checking

- Even if a `scanf` operates successfully, the values read might still be invalid.
- For example, grades are typically integers in the range 0–100. In a program that inputs such grades, you should *validate the grades by using range checking* to ensure that they are values from 0 to 100.
- You can then ask the user to re-enter any value that's out of range.
- If a program requires *inputs from a specific set of values* (e.g., non-sequential product codes), you can *ensure that each input matches a value in the set*.

Exercises

- Describe the problem of the following code, and show how to fix it. Suppose x , y , and z are declared as integers.

```
if ( x < 2 )
    if ( y > 4 )
        z = x + y;
else
    z = x - y;
```

Exercises

- Write a complete C program to sum up the odd integers from 1 to 100.
- Write C code for the following scenarios using logical operators.
 - If x is greater than 5 or x is less than 3 then set z to zero
 - If x is greater than 3 and y is less than 5 then set z to one

Exercises

- Write C code to print the integers from 1 to 20 using for, while, and do...while statements.
- Give a short description of the return value of scanf. How to use it to ensure secure programming? Use `scanf ("%d %d", &x, &y)` to explain your answer.

Exercises

- What is the function of rules of operator precedence? What is operator associativity? Explain the problem of order of evaluation of operands in C.
- Explain why controlling counting loops with floating-point values is dangerous.

Exercises

- What will occur if a case in the switch selection statement has no break?
- Explain the effect of short-circuit evaluation in C.

Exercises

- What will happen if a programmer intentionally creates a for-loop with the for header:
`for ( ; ; ) ?`
- What are lvalue and rvalue? What will happen if we assign value to an rvalue?

Exercises

- Give $x = 12$, $y = 6$, and $z = 1$. What is the output of x , y , and z after executing the following code?

```
if ( ( x > 6 ) || ( y = 1 ) ) {  
    z = 4;  
}
```

- What is off-by-one error?